

XML

Schéma XML

Des DTD au schémas

- Les DTD présentent l'inconvénient d'être pauvre en possibilités de contrôle.
- Les espaces de noms sont difficilement gérables car il faut les intégrer dans la grammaire avec les préfixes.
- Ce qui est contraire à l'esprit des espaces de noms.
- Un schéma W3C est une grammaire définie dans un formalisme XML.

Caractéristiques des schémas

- Quelques caractéristiques des schémas :
 - gestion des espaces de noms ;
 - types de base riches et extensibles ;
 - réutilisation par importation et héritage ;
 - davantage de souplesse dans les cardinalités.

Les différentes formes de type

- Dans un schéma, les types sont fondamentaux.
- Ils ont la caractéristique d'être disponibles à la fois pour les **éléments** et les **attributs**.

Les contenus des éléments

- les contenus possibles pour un élément :
 - **Vide** (entraîne EMPTY avec une DTD) : pas de contenu.
 - **Simple** (entraîne (#PCDATA) avec une DTD) : du texte.
 - **Complexe** : un ou plusieurs éléments.
 - **Mixte** : un mélange d'éléments et de texte.

Les types des éléments

- À partir de ces contenus possibles, il ressort dans les schémas 2 types :
 - **Simple** : du texte pour attribut ou élément ;
 - **Complexe** : avec au moins un attribut ou un élément fils.

Le typage des attributs

- il s'agit d'un typage simple
- même principe que le typage d'un élément ne contenant que du texte et sans attributs.

Pour résumer

- **Pas de type :**
 - élément sans contenu et sans attribut.
- **Type simple :**
 - attribut ;
 - élément avec du texte seulement.
- **Type complexe :**
 - tous les autres cas d'usage de l'élément.

Quelques exemples :

`<auteur/>` : pas de type

`<auteur id="10"/>` : type simple

`<auteur>DIOP</auteur>` : type simple

`<auteur id="10"/>` : type complexe

`<auteur id="10">DIOP</auteur>` : type complexe

<code><auteur></code>	}	: type complexe
<code><titre>M.</titre></code>		
<code><nom>DIOP</nom></code>		
<code></auteur></code>		

Les définitions globales et locales

- Dans une DTD toutes les définitions sont globales.
- Il n'est pas possible d'utiliser plusieurs définitions pour un même nom d'élément dans une DTD.
- L'exemple suivant ne peut pas être géré avec une DTD :

```
<document>  
  <titre>...</titre>  
  <auteur>  
    <titre>...</titre>  
  </auteur>  
</document>
```


Les définitions globales et locales

- Dans un schéma, nous avons le choix, soit de créer une définition globale, soit de créer une définition locale.
- Une définition globale concerne également d'autres parties du schéma, comme les types, groupes, attributs...
- Une définition locale est contextuelle à un élément.

L'assignation d'un schéma à un document XML

- Le document XML est considéré comme une instance du schéma.
- Pour l'assignation d'un schéma à un document XML, on l'insère dans le dernier des attributs supplémentaires sur l'élément racine.

L'assignation d'un schéma à un document XML

- Ces attributs devront appartenir à l'espace de noms <http://www.w3.org/2001/XMLSchema-instance>.
- Il existe deux formes de liens pour désigner la localisation du schéma à employer:
 - **noNamespaceSchemaLocation**, cas ignorant les espaces de noms du document,
 - **schemaLocation**, cas prenant en compte les espaces de noms du document.

L'assignation d'un schéma à un document XML

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<personnel xmlns:xsi="http://www.w3.org/2001/XMLSchema-  
instance"
```

```
  ➡xsi:noNamespaceSchemaLocation="personal.xsd">
```

```
...
```

```
</personnel>
```


Les catégories de type simple

- Les types applicables aux **attributs** et au **contenu d'éléments simples** (sans attribut) sont divisés en 2 catégories :
 - **Types normalisés** : tous les caractères blancs (tabulation, retour chariot...) sont remplacés par un caractère unique.
 - **Types compactés** : les occurrences de blancs en tête et queue sont supprimées. Les séries contiguës de blancs sont remplacées par un blanc.

Répartition des types de base

Non normalisé et non compacté	Normalisé et non compacté	Normalisé et compacté
xs:string	xs:normalized String	xs:byte xs:int xs:long xs:float xs:double xs:boolean xs:date xs:time xs:token ...

Utilisation de type simple :

- Exemple :
 - `<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">`
 - `<xs:element name="titre" type="xs:string" />`
 - `<xs:element name="numchapitre" type="xs:int" />`
 - `</xs:schema>`
- 2 documents possibles, valides :
 - `<titre>Un titre</titre>`
 - et
 - `<numchapitre>10</numchapitre>`

Les types pour les chaînes de caractères

- **xs:string** : le plus simple sans normalisation ni compactage ;
- **xs:normalizedString** : la forme normalisée et non compactée ;
- **xs:token** : la forme normalisée et compactée ;
- **xs:language** : les codes de langue ;
- **xs:NMTOKEN** : pas de blanc ;
- **xs:Name** : forme de NMToken commençant par une lettre ;
- **xs:ID** : chaîne unique dans le document ;
- **xs:IDREF** : chaîne ayant pour valeur un ID;
- **xs:anyURI** : la valeur de l'URI utilisera des caractères ASCII.

Les types pour les dates et heures

- xs:dateTime,
- xs:date,
- xs:time,
- xs:duration,
- xs:gYearMonth,
- xs:gYear,
- xs:gDay,
- xs:gMonth ;

Format lexical

- Format lexical complet de `xs:dateTime` : `CCYY-MM-DDThh:mm:ssZff`.
- Quelques exemples du type `xs:dateTime` :
 - `2006-08-21T12:07:00` : fuseau horaire inconnu ;
 - `2006-08-21T12:07:00+02:00` : décalage de 2 heures avec le TU ;
 - `2006-08-21T12:07:00Z2` : décalage de 2 heures avec le TU.

Les types numériques

- **xs:int (32 bits)** : entier, également **xs:unsignedInt**.
- **xs:long (64 bits)**, **xs:short (16 bits)**, **xs:byte (8 bits)** : entier ; il existe également **xs:unsignedLong**, **xs:unsignedShort**, **xs:unsignedByte**.
- **xs:float (32 bits)** : flottant simple précision ; **xs:double (64 bits)** : flottant double précision.
- **xs:decimal**: nombre sans limitation ;
- **xs:integer** : forme de **xs:decimal** sans la partie décimale.
- **xs:nonPositiveInteger** : entier négatif avec le zéro ; **xs:negativeInteger** : entier négatif sans le zéro.
- **xs:boolean** : true ou false ou 1 ou 0.

La création de nouveaux types simples

- **La restriction** : comme son nom l'indique, on crée un sous-type en limitant certaines caractéristiques (par exemple, la longueur d'une chaîne). Les limitations sont définies avec des facettes.
- **L'union** : l'union sert à autoriser plusieurs types, ce qui permet d'offrir des alternatives, comme un attribut ayant pour valeur soit un entier soit une constante.
- **La liste** : il s'agit d'un ensemble de valeurs de même type séparées par un blanc. Il est également possible d'appliquer une restriction sur une liste, notamment pour en borner la taille.

La restriction

- Une restriction fait intervenir des balises qui vont réduire l'espace des valeurs possibles pour un type de base.
- Ces balises ont toutes un attribut value pour définir l'ampleur de la limitation.
- **Exemple 1**

```
<xs:simpleType name="chaineCompacte">  
  <xs:restriction base="xs:String">  
    <xs:whiteSpace value="collapse"/>  
  </xs:restriction>  
</xs:simpleType>
```

La restriction

- `<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">`
 - `<xs:simpleType name="couleurType">`
 - `<xs:restriction base="xs:string">`
 - `<xs:enumeration value="bleu"></xs:enumeration>`
 - `<xs:enumeration value="blanc"></xs:enumeration>`
 - `<xs:enumeration value="rouge"></xs:enumeration>`
 - `</xs:restriction>`
 - `</xs:simpleType>`
 - `<xs:element name="crayon">`
 - `<xs:complexType>`
 - `<xs:attribute name="couleur" type="couleurType"/>`
 - `</xs:complexType>`
 - `</xs:element>`
- `</xs:schema>`

L'union

- L'union sert à combiner plusieurs types simples ;
- il suffira qu'une valeur corresponde à l'un des types pour que la valeur soit correcte.
- Exemple 1 :

```
<xs:simpleType>  
  <xs:union memberTypes="xs:int xs:boolean"/>  
</xs:simpleType>
```

L'union

- Exemple 2 :

```
<xs:simpleType>
  <xs:union>
    <xs:simpleType>
      <xs:restriction base="xs:int"/>
    </xs:simpleType>
    <xs:simpleType>
      <xs:restriction base="xs:boolean"/>
    </xs:simpleType>
  </xs:union>
</xs:simpleType>
```


La liste

- L'autre forme de dérivation d'un type simple est la liste de valeurs. Les valeurs d'une liste sont séparées par un blanc et il est impossible d'utiliser un autre séparateur.
- Exemple 1 :

```
<xs:simpleType>  
  <xs:list itemType="xs:int" />  
</xs:simpleType>
```
- La liste 10 20 333 4 3 est donc bien cohérente avec ce type.

L'utilisation des types complexes

- Dans un schéma, le type complexe nécessite toujours une balise `complexType`.
- Il peut être global et donc associé à un nom ou bien être local à un élément.

Premier connecteur : la séquence

- La séquence caractérise des éléments fils présents dans un ordre donné.
- Exemple :
- `<xs:element name="plan">`
 - `<xs:complexType>`
 - `<xs:sequence>`
 - `<xs:element name="auteurs" type="xs:string"/>`
 - `<xs:element name="chapitres" type="xs:string"/>`
 - `</xs:sequence>`
 - `</xs:complexType>`
- `</xs:element>`

```
<!ELEMENT plan (auteurs,chapitres)>
<!ELEMENT auteurs (#PCDATA)>
<!ELEMENT chapitres (#PCDATA)>
```

Premier connecteur : la séquence

- Nous aurions pu également l'écrire avec un type global (par exemple planType) :
- `<xs:complexType name="planType">`
 - `<xs:sequence>`
 - `<xs:element name="auteurs" type="xs:string"/>`
 - `<xs:element name="chapitres" type="xs:string"/>`
 - `</xs:sequence>`
- `</xs:complexType>`
- `<xs:element name="plan" type="planType"/>`

Deuxième connecteur : le choix

- Le choix, comme son nom l'indique, étend les possibilités de constituer des documents XML valides en proposant des alternatives d'éléments.
- Exemple :

```
<xs:element name="plan">
  <xs:complexType>
    <xs:choice>
      <xs:element name="sections" type="xs:string"/>
      <xs:element name="chapitres" type="xs:string"/>
    </xs:choice>
  </xs:complexType>
</xs:element>
```

```
<!ELEMENT plan (sections | chapitres)>
<!ELEMENT sections (#PCDATA)>
<!ELEMENT chapitres (#PCDATA)>
```

Dernier connecteur : tout

- Le connecteur all est propre au schéma et caractérise un ensemble d'éléments de présence obligatoire mais sans contrainte sur l'ordre (toutes les permutations sont donc valides).
- Exemple :**

```
<xs:element name="plan">
  <xs:complexType>
    <xs:all>
      <xs:element name="auteur" type="xs:string"/>
      <xs:element name="chapitres" type="xs:string"/>
    </xs:all>
  </xs:complexType>
</xs:element>
```

```
<!ELEMENT plan ((auteur,chapitres)|(chapitres,auteur))>
<!ELEMENT sections (#PCDATA)>
<!ELEMENT chapitres (#PCDATA)>
```


La limitation des quantités d'éléments (les cardinalités)

- Tout comme dans les DTD, les cardinalités sont possibles sur les éléments de connecteur ou sur les connecteurs eux-mêmes (jouant le rôle des parenthèses dans une DTD).
- Ces cardinalités sont positionnées par les attributs minOccurs et maxOccurs. L'infini est caractérisé par la chaîne unbounded.

La limitation des quantités d'éléments (les cardinalités)

- Pour faire le parallèle avec les DTD, nous retrouvons l'équivalent des opérateurs ?, + et * avec :
 - ? : minOccurs="0" maxOccurs="1" ;
 - + : minOccurs="1" maxOccurs="unbounded" ;
 - * : minOccurs="0" maxOccurs="unbounded".
- Lorsqu'on ne précise rien, minOccurs et maxOccurs ont la valeur 1.

La limitation des quantités d'éléments (les cardinalités)

- Exemple :

```
<xs:element name="plan">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="auteur" type="xs:string" maxOccurs="unbounded"/>
      <xs:element name="chapitre" type="xs:string" minOccurs="2"
        maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

La limitation des quantités d'éléments (les cardinalités)

- Exemple :

```
<xs:element name="plan">
  <xs:complexType>
    <xs:sequence minOccurs="2" maxOccurs="3">
      <xs:element name="auteur" type="xs:string"/>
      <xs:element name="chapitre" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```


Définition d'un attribut dans un type complexe

- L'attribut implique la présence d'un type complexe.
- Il est toujours placé en dernière position.
- L'attribut en lui-même, ne contenant que du texte, est un type simple.
- L'attribut peut être global et donc réutilisable au sein de plusieurs définitions de type complexe.

Définition d'un attribut dans un type complexe

- Exemple :

```
<xs:element name="personne" >
```

```
  <xs:complexType>
```

```
    ...
```

```
    <xs:attribute name="nom" type="xs:string"/>
```

```
  </xs:complexType>
```

```
</xs:element>
```


Définition d'un attribut dans un type complexe

```
<xs:schema ...>
  <xs:attribute name="nom">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:minLength value="5"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:attribute>
  <complexType name="monType">
    <xs:attribute ref="nom"/>
  </complexType>
</xs:schema>
```

FIN.
